PHASE 4 · CI/CD SUPPLY CHAIN · ADVANCED

## Chapter 17

# Complete CI/CD Pipeline: End-to-End Orchestration

*From code commit to a production-ready, fully-scanned container image — every security gate, scan, and approval working together.*

| 11 | 5 | 3–4h |
|---|---|---|
| PIPELINE STAGES | SECURITY GATES | TIME ESTIMATE |

# Chapter Goal

*Orchestrate every Phase 4 component into one complete, secure software delivery pipeline.*

### Complete CI/CD pipeline orchestrated

All 11 stages wired together into one flow.

### Automatic remediation workflows

Findings tracked and routed automatically.

### All tools working together seamlessly

GitLab, SonarQube, Harbor, DefectDojo, Vault.

### Deployment gates and approval flows

Human checkpoints before anything ships.

### Every security gate enforced

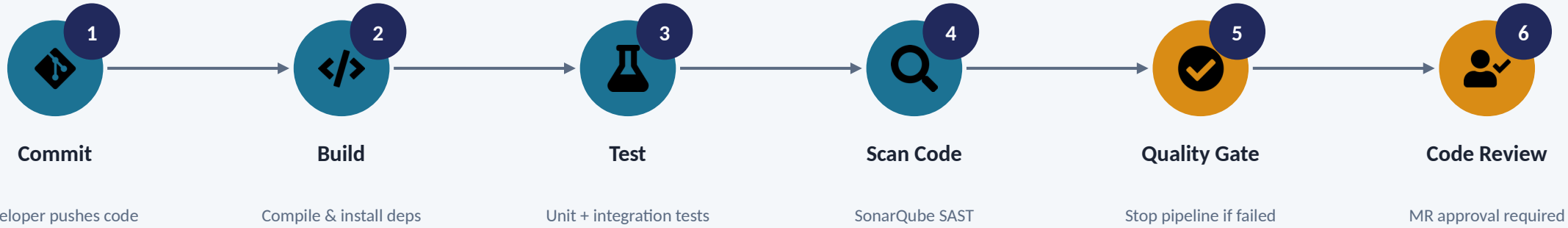
SAST, dependency, image, and compliance scans.

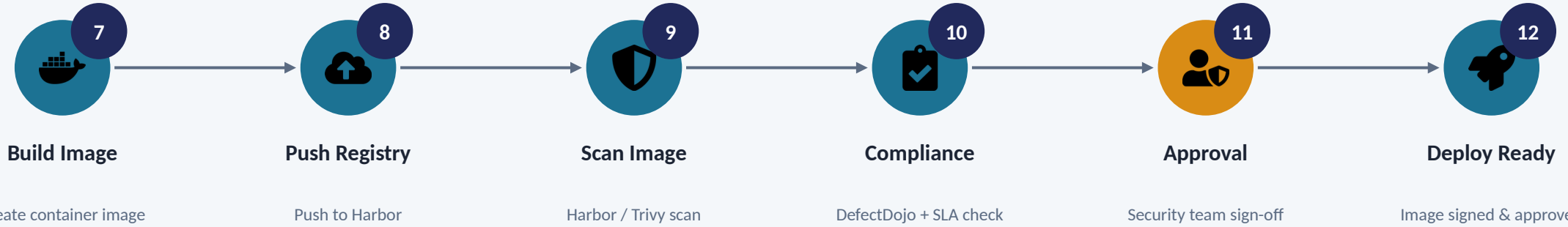
**Time Estimate**

3–4 hours to complete this chapter

# 17.1 Complete Pipeline Flow

*Commit to production-ready image — eleven stages, five enforced gates.*




Automated stage

Gate / human checkpoint

# 17.2 Build, Test & Static Scan

*.gitlab-ci.yml — Stages 1–3*

### Stage 1 · Build

- Image: python:3.11
- pip install -r requirements.txt
- Compile app.py & tests/*.py
- Artifacts kept 1 hour

### Stage 2 · Test

- pytest with coverage (xml + term)
- Cobertura coverage report
- Runs only on merge requests
- Coverage regex captures TOTAL %

### Stage 3 · Scan Code (SAST)

- sonar-scanner CLI image
- Scans full source + coverage report
- allow_failure: false — cannot skip
- Runs on MRs and main branch

# 17.2 Quality Gate & Code Review

*Stage 4 blocks bad code automatically. Stage 5 requires a human.*

## Stage 4 · Quality Gate

- Query SonarQube's project_status API
- Parse status with jq -r '.projectStatus.status'
- If status ≠ "OK" → exit 1, pipeline stops
- Runs on merge requests and main branch

```
❌ Quality gate FAILED  →  exit 1

✅ Quality gate PASSED  →  continue
```

*Terminal output pattern from the gate check script*

## Code Review (Manual Gate)

*Happens in the GitLab merge request UI — not a pipeline stage in the YAML, but a hard organizational gate:*

| | |
|---|---|
| **john** | ✅ approved |
| **alice** | ⏳ pending |
| **bob** | ⏳ pending |

Cannot merge without required approvals — this is the human checkpoint before any image gets built.

# 17.2 Build, Push & Scan the Image

*Stages 6–8 — only run on main/master, after merge*

### Stage 6 · Build Image

*docker:20.10 + dind service*

```
docker build -t $IMAGE_NAME:
$CI_COMMIT_SHA .


Also tags :latest
```

✅ **Docker image built: $IMAGE_NAME: $CI_COMMIT_SHA**

### Stage 7 · Push to Harbor

*Authenticate & publish*

```
docker login to $REGISTRY_URL


Push both SHA and :latest tags
```

✅ **Image pushed to Harbor**

### Stage 8 · Scan Image

*Vulnerability gate*

```
Query Harbor scan_overview.critical


critical > 0 → exit 1, pipeline
fails
```

```
if [ "$VULNERABILITIES" -gt 0 ]; then
exit 1; fi
```

✅ **Image scan passed (no critical vulnerabilities)**

# 17.2 Compliance Check & Manual Approval

*Stages 9–11 — nothing reaches deployment without a person signing off*

### Stage 9 · Compliance Check (DefectDojo)

- **DefectDojo findings**

  SonarQube results imported as a finding via the /findings/ API

- **SLA check**

  Findings tracked against remediation SLA timelines

- **Critical vulnerability check**

  Blocks progress if unresolved criticals remain

```
POST $DEFECTDOJO_URL/findings/  { product, scan_type, scanner_name, file }
```

### Stage 10 · Image Approval

```
when: manual
```

Security team must manually trigger this job before the pipeline can proceed — no automatic pass-through.

### Stage 11 · Ready for Deployment

Image signed and approved — cleared for staging or production deployment.

# 17.3 Secrets Management in the Pipeline

*Fetch credentials from Vault at pipeline runtime — never hard-code them*



### Why Vault, not variables?

- Secrets are fetched live from Vault, never stored in the .gitlab-ci.yml
- before_script runs the fetch on every job, before the main script
- X-Vault-Token authenticates the request to the Vault API
- jq extracts just the needed field from the JSON response

```
before_script:

# Database password

export DB_PASSWORD=$(curl -s \

  -H "X-Vault-Token: $VAULT_TOKEN" \

  $VAULT_ADDR/v1/secret/data/database/mysql |

  jq -r '.data.data.password')


# Harbor registry credentials

export HARBOR_CREDENTIALS=$(curl -s \

  -H "X-Vault-Token: $VAULT_TOKEN" \

  $VAULT_ADDR/v1/secret/data/registry/harbor |

  jq -r '.data.data')
```

# 17.5 Notifications & Alerting

*Every job reports its outcome to Slack via after_script*

### ✅ PASSED

Green attachment · job stage, commit SHA, and branch attached as fields

### ❌ FAILED

Red ("danger") attachment · same fields, sent to the same webhook

*Delivered to $SLACK_WEBHOOK_URL — one shared webhook for the whole pipeline*

```
after_script:
  - |
    if [ "$CI_JOB_STATUS" = "success" ]; then
      STATUS="✅ PASSED"; COLOR="good"
    else
      STATUS="❌ FAILED"; COLOR="danger"
    fi
    curl -X POST -H 'Content-type: application/json' \
      --data '{"attachments":[{
        "color": "'$COLOR'",
        "title": "'$CI_PROJECT_NAME' - '$CI_JOB_STAGE'",
        "text": "'$STATUS'"
      }]}' \
      $SLACK_WEBHOOK_URL
```

# 17.6 Advanced Workflows

*Progressive rollout and safety net for deployments*

## Canary Deployment




*Traffic shifted gradually while monitoring for errors*

- Deploy to 10% of traffic first
- Monitor for errors for 15 minutes
- If stable, increase to 50%, then 100%

```
kubectl set image deployment/myapp-canary \
  myapp=$IMAGE_NAME:$CI_COMMIT_SHA --record
```

## Automatic Rollback

*A manual safety-net job — always available if a deployment goes wrong:*

- Stage: deploy
- when: manual — triggered on demand
- Reverts to the previous stable release

```
kubectl rollout undo deployment/myapp
echo "Rolled back to previous version"
```

*Keep this job ready in every environment — speed matters during an incident.*

# 17.7 Performance & Optimization

*Keep the pipeline fast without cutting security corners*

## Cache Dependencies

Cache pip packages keyed on requirements.txt so unchanged dependencies skip re-download.

```
cache:
  paths:
    - .cache/pip
  key:
    files:
      - requirements.txt

before_script:
  - export PIP_CACHE_DIR=".cache/pip"
  - pip install -r requirements.txt
```

## Parallel Test Matrix

Run the same test job across multiple Python versions simultaneously.

**Python 3.9** | **Python 3.10** | **Python 3.11**

```
test_multiple_versions:
  stage: test
  parallel:
    matrix:
      - PYTHON_VERSION: ["3.9","3.10","3.11"]
  image: python:$PYTHON_VERSION
  script:
    - pytest tests/
```

# 17.8 Chapter Summary

### End-to-end pipeline

All tools working together

### Security gates

Every stage validates

### Approval workflows

Human checkpoints

### Automated remediation

Findings tracked automatically

### Real-time status

Visibility into pipeline

### Integration

All Phase 4 tools orchestrated

### 💡 Key Takeaway — Complete Automation with Safety

Fully automated, but never unsupervised — no code reaches production without code review, quality gate, security scan, image scan, and manual approval.